Practical 07

1.

AIM :

Form validator: define @NotBlank and @MaxLength(int) for fields and annotate a SignupForm. Write a checker that uses reflection to read each field's annotations and returns the list of validation errors for a given object. **CODE :** import java.lang.annotation.*; import java.lang.reflect.*; import java.util.*;

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface NotBlank {
}

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface MaxLength {
   int value();
}
class SignupForm {
@NotBlank
```

```
    String name;

    @MaxLength(10)
    String username;

    SignupForm(String n, String u) {
name = n;         username = u;
    }
}

public class Main {

    static ArrayList<String> validate(Object obj) throws
Exception {
        ArrayList<String> errors = new ArrayList<>();

        Field[] fields = obj.getClass().getDeclaredFields();
for (Field f : fields) {
            f.setAccessible(true);
Object value = f.get(obj);
```

```
            if (f.isAnnotationPresent(NotBlank.class)) {
if (value == null || value.toString().trim().isEmpty())
{
                    errors.add(f.getName() + " cannot be
blank");
                }
            }

            if (f.isAnnotationPresent(MaxLength.class)) {
                MaxLength m =
f.getAnnotation(MaxLength.class);

                if (value != null && value.toString().length() >
m.value()) {
                    errors.add(f.getName() + " is too long");
                }
            }
        }
        return errors;
    }

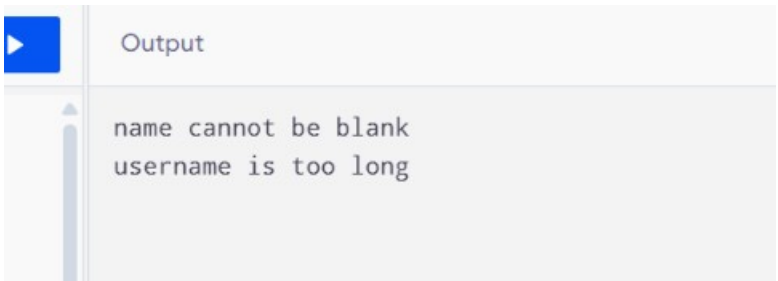
    public static void main(String[] args) throws Exception
{
```

```
        SignupForm form = new SignupForm("",
"abcdefghijkl");


        ArrayList<String> errors = validate(form);


        if (errors.isEmpty()) {
            System.out.println("Valid form");
        } else {
            for (String s : errors) {
                System.out.println(s);
            }
        }
    }
}
```

OUTPUT :



2.

AIM :

Mini test runner: define a @Run annotation and mark some no-arg methods of a class with it. Use reflection to find and invoke only the @Run methods and report how many ran — the idea behind how JUnit finds @Test methods.

CODE :

```
import java.lang.annotation.*;
import java.lang.reflect.*;


@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@interface Run {
}


class MyTests {


    @Run
    public void testOne() {
        System.out.println("Test One executed");
    }


    @Run
```

```
    public void testTwo() {
        System.out.println("Test Two executed");
    }

    public void normalMethod() {
        System.out.println("Normal method");
    }
}

public class Main {
    public static void main(String[] args) throws Exception
{

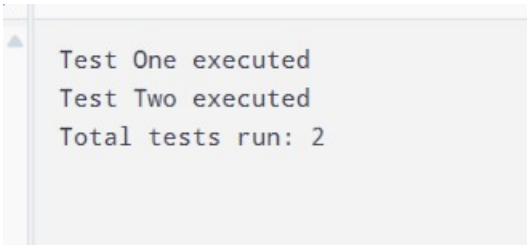
        MyTests t = new MyTests();
int count = 0;
        Method[] methods =
t.getClass().getDeclaredMethods();

        for (Method m : methods) {
            if (m.isAnnotationPresent(Run.class)) {
                m.invoke(t);
count++;
```

```
        }
    }

    System.out.println("Total tests run: " + count);
  }
}
```

OUTPUT :



3.

AIM :

(For Advanced Learners – home) Define a @Column(name) annotation and, by reflection, populate an object's fields from a header row and a data row by matching column names; decide how to handle a missing column.

CODE :

```
import java.lang.annotation.*;
import java.lang.reflect.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface Column {
    String name();
}

class Student {
    @Column(name = "roll")
    int roll;

    @Column(name = "name")
    String name;
}

public class Main {

    public static void main(String[] args) throws Exception
{
```

```
        String[] header = {"roll", "name"};
        String[] data = {"101", "Grishma"};

        Student s = new Student();

        Field[] fields = s.getClass().getDeclaredFields();

        for (Field f : fields) {
            if (f.isAnnotationPresent(Column.class)) {

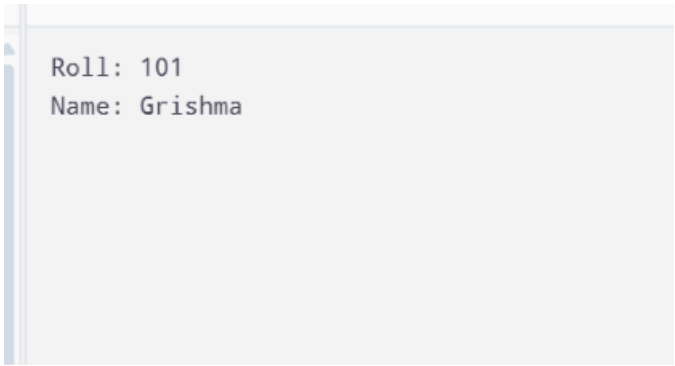
Column c = f.getAnnotation(Column.class);

                for (int i = 0; i < header.length; i++) {
if (header[i].equals(c.name())) {
                        f.setAccessible(true);
                        if (f.getType() == int.class) {
                            f.setInt(s, Integer.parseInt(data[i]));
                        } else {
                            f.set(s, data[i]);
                        }
                    }
```

```
            }
        }
    }

    System.out.println("Roll: " + s.roll);
    System.out.println("Name: " + s.name);
  }
}
```

OUTPUT :



LEARNING OUTCOME :

1. Understand serialization and deserialization in Java.
2. Save and load objects using ObjectOutputStream and ObjectInputStream.
3. Read and write files using Java NIO.
4. Maintain transaction logs and generate reports.
5. Apply file handling and persistence concepts to the MiniBank project.